

Findr

Software Requirements Specification — Campus Lost & Found Board

Document Version	1.0
Group	Group 7
Institution	Zeal College of Engineering & Research, Narhe, Pune
Project Type	College Internship Project
Project Duration	15 Days
Date	July 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for Findr, a web application that allows students to report, browse, and claim lost or found items within their college campus. It is intended for use by the developer, project guide, and evaluators as a reference for the system's intended functionality and scope.

1.2 Scope

The system allows a student to create an account using their college-issued ZPRN ID, report an item as lost or found, browse all reported items on a shared board, submit a claim on an item with a message to the reporter, and allow the original reporter to approve or reject claims made on their reported items. The system is built as a project-scale web application intended for demonstration and internal college use, not for large-scale production deployment.

1.3 Intended Audience

College students as end users, project evaluators/guides reviewing the submission, and the developer for future reference and extension of the project.

1.4 Definitions and Abbreviations

Term	Definition
ZPRN ID	Unique student identification number issued by the college, used to identify a user
SRS	Software Requirements Specification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete — basic data operations

1.5 Technology Stack

Layer	Technology
Backend / Web Server	Node.js with Express.js
Database	MySQL
Frontend	HTML, CSS, vanilla JavaScript

2. Overall Description

2.1 Product Perspective

Findr is a standalone web application consisting of a REST API backend and a browser-based frontend. It is not integrated with any existing college ERP or identity system; ZPRN ID is used only as

a unique identifier field within this system's own database.

2.2 Product Functions

- Registering a student profile using ZPRN ID
- Reporting an item as lost or found, with description, category, location, and contact info
- Browsing all reported items on a shared, public board
- Submitting a claim with a message on an item believed to belong to the claimant
- Reviewing and approving/rejecting claims made on items the user has reported

2.3 User Characteristics

End users are college students with basic familiarity with web applications and no special technical training required to use the system.

2.4 Constraints

- Built within a 15-day project timeline as a college internship deliverable
- Uses MySQL as the database, requiring a running MySQL server in the deployment environment
- No integration with official college identity verification for ZPRN ID at present

2.5 Assumptions and Dependencies

It is assumed that each student's ZPRN ID is unique. The system depends on Node.js, the associated npm packages (Express, mysql2), and a running MySQL server being installed in the deployment environment.

3. Functional Requirements

ID	Requirement	Description
FR-1	User Registration	System shall allow a new user to register using name and ZPRN ID. ZPRN ID must be unique.
FR-2	Report Item	A user shall be able to report an item with name, status (lost/found), category, location, contact info, and description.
FR-3	Browse Board	Any visitor shall be able to view all reported items along with the reporter's name and contact info.
FR-4	Submit Claim	A user shall be able to submit a claim with an optional message and contact info on any unclaimed item.
FR-5	View Claims	The system shall display all claims submitted on a given item, including the date each claim was made.
FR-6	Approve/Reject Claim	Only the original reporter of an item shall be able to approve or reject a claim on that item.
FR-7	Auto-update Item Status	When a claim is approved, the associated item's status shall automatically update to 'claimed'.

4. External Interface Requirements

4.1 User Interface

The frontend is a single-page browser interface with three sections: Report Item, The Board (browse all items), and My Items (view and resolve claims on the user's own reported items).

4.2 Hardware Interfaces

No special hardware is required. The system runs on any standard PC capable of running Node.js and a modern web browser.

4.3 Software Interfaces

Backend: Node.js runtime with Express framework. Database: MySQL, accessed via the mysql2 library. Frontend communicates with the backend over HTTP using JSON, via the Fetch API.

4.4 Communication Interfaces

The frontend and backend communicate over HTTP on localhost (port 3000 by default) during development.

5. Non-Functional Requirements

Category	Requirement
----------	-------------

Security	User-submitted data shall be validated before being stored to reduce the risk of malformed or malicious input.
Usability	The interface shall be simple enough for a first-time user to register and report an item without external instructions.
Performance	The system shall return item listings within 1 second under normal demo-scale load (fewer than 100 concurrent users).
Reliability	Core data (users, items, claims) shall persist across server restarts via the MySQL database.
Maintainability	Code is organized into separate files for database schema (db.js), server logic (server.js), and frontend (index.html) for ease of modification.
Portability	The application can run on any OS supporting Node.js and MySQL (Windows, macOS, Linux) with no code changes.

6. System Data Model

6.1 Entities and Relationships

The system uses three related tables: Users, Items, and Claims. A user can report many items (one-to-many). An item can receive many claims (one-to-many). Each claim belongs to exactly one item and one claimant.

6.2 Users Table

Field	Type	Notes
id	INT	Primary key, auto-increment
name	VARCHAR(100)	Required
zprnId	VARCHAR(50)	Unique, required — identifies the user
createdAt	DATETIME	Defaults to current timestamp

6.3 Items Table

Field	Type	Notes
id	INT	Primary key, auto-increment
itemName	VARCHAR(150)	Required
description	TEXT	Optional
category	VARCHAR(50)	Optional
location	VARCHAR(150)	Optional
imageUrl	VARCHAR(255)	Optional — path or link to a photo of the item
contactInfo	VARCHAR(150)	Contact details for the reporter (e.g. phone or email)
status	VARCHAR(20)	'lost', 'found', or 'claimed' — defaults to 'lost'
reportedBy	INT	Foreign key → users.id
dateReported	DATETIME	Defaults to current timestamp

6.4 Claims Table

Field	Type	Notes
id	INT	Primary key, auto-increment
itemId	INT	Foreign key → items.id
claimantId	INT	Foreign key → users.id
claimMessage	TEXT	Optional message from claimant

contactInfo	VARCHAR(150)	Contact details for the claimant (e.g. phone or email)
status	VARCHAR(20)	'pending', 'approved', or 'rejected' — defaults to 'pending'
dateClaimed	DATETIME	Date and time the claim was submitted — defaults to current timestamp

7. Future Scope / Known Limitations

- Add a login/authentication layer if stronger identity verification is needed in future versions
- Integrate with official college systems for ZPRN ID verification
- Introduce a reward points system to encourage students to report and return found items, with points redeemable for small campus incentives